

COOCHBEHAR GOVERNMENT ENGINEERING COLLEGE

DIGITAL COMMUNICATION LABORATORY(EC-592)

Experiment 5 — DPCM and Delta Modulation

Title-Simulation and Performance Analysis of DPCM and Delta Modulation

Name: Murshid Alam

University Roll:34900324033

Department: Electronics and communication engineering

Session:2026-2027 Year:3rd semester:5th

1.Aim

To implement and analyze Differential Pulse Code Modulation (DPCM) and Delta Modulation (DM) using MATLAB, and to study the effects of prediction error, quantization error, step size, and input frequency on the reconstructed signal.

2. Objectives

- To implement a first-order DPCM predictor.
- To compare PCM and DPCM quantization errors.
- To implement Delta Modulation for different step sizes.
- To study the effect of small, moderate and large step sizes.
- To identify:
 - Granular noise
 - Slope-overload distortion
- To test slowly and rapidly varying input signals.
- To plot:
 - Original and predicted samples
 - Prediction error
 - Delta staircase

- MSE versus step size
- To verify that every DM output step changes by exactly $+\Delta$ or $-\Delta$.

3. Theory

A. DPCM

In conventional PCM, every sample of the input signal is quantized and encoded independently.

DPCM reduces the number of bits required by exploiting the correlation between adjacent samples.

For a first-order predictor,

$$\hat{x}[n] = x[n - 1]$$

where:

- $x[n]$ = current input sample
- $\hat{x}[n]$ = predicted sample

The prediction error is

$$e[n] = x[n] - \hat{x}[n]$$

The prediction error generally has a smaller amplitude range than the original signal. Therefore, it can be quantized using fewer bits.

DPCM process

$$x[n] \rightarrow \boxed{\text{Predictor}} \rightarrow e[n] \rightarrow \boxed{\text{Quantizer}} \rightarrow \text{Encoded } e[n]$$

At the receiver:

$$\text{Quantized Error} \rightarrow \boxed{\text{Reconstruction}} \rightarrow \hat{x}_r[n]$$

B. Delta Modulation

Delta Modulation is a simplified form of DPCM in which the difference signal is represented using only **one bit per sample**.

The reconstructed signal changes only by a fixed step size:

$$x_r[n] = x_r[n - 1] + \Delta$$

or

$$x_r[n] = x_r[n - 1] - \Delta$$

Therefore, every output step must be exactly:

$$\boxed{+\Delta \quad \text{or} \quad -\Delta}$$

This is the **mandatory validation** specified in the experiment.

4. Types of Delta-Modulation Distortion

4.1 Granular Noise

If the step size Δ is too large compared with the signal variation, the staircase cannot closely follow the signal near slowly varying portions.

This produces **granular noise**.

4.2 Slope-Overload Distortion

If Δ is too small, the staircase cannot follow a rapidly changing input signal.

The input changes faster than the maximum staircase slope:

$$\left| \frac{dx}{dt} \right| > \Delta f_s$$

Therefore, the reconstructed signal falls behind the input.

This is called **slope-overload distortion**.

5.MATLAB Program

%% EXPERIMENT 5: DPCM AND DELTA MODULATION

% Digital Communication Laboratory

% First-order DPCM + Delta Modulation

**% Demonstrates prediction, quantization,
reconstruction,**

% granular noise, slope overload and +/- Delta validation.

Clear;

Close all;

Clc;

%%

=====

=====

% 1. SIGNAL PARAMETERS

%

=====

=====

Fs = 1000; % Sampling frequency (Hz)

T = 1; % Signal duration (seconds)

T = 0:1/fs:T-1/fs;

N = length(t);

% Slowly varying input

F_slow = 5;

X_slow = sin(2*pi*f_slow*t);

% Rapidly varying input

F_fast = 80;

X_fast = sin(2*pi*f_fast*t);

% Main input signal

F0 = 10;

X = sin(2*pi*f0*t);

%%

=====

=====

% 2. FIRST-ORDER DPCM PREDICTION

%

=====
=====

X_pred = zeros(size(x));

% First sample prediction

X_pred(1) = 0;

% First-order predictor

For n = 2:N

X_pred(n) = x(n-1);

End

% Prediction error

Prediction_error = x - x_pred;

%%

=====

=====

% 3. DPCM QUANTIZATION

%

=====

=====

% Quantizer parameters

L = 16; % Number of quantization levels

Xmin = min(prediction_error);

Xmax = max(prediction_error);

Delta_q = (xmax – xmin)/(L-1);

% Quantization

Q_error = round((prediction_error-xmin)/delta_q);

Q_error = q_error*delta_q + xmin;

% DPCM reconstruction

X_dpcm = zeros(size(x));

X_dpcm(1) = x(1);

For n = 2:N

X_dpcm(n) = x_dpcm(n-1) + q_error(n);

End

%%

=====

=====

% 4. PCM QUANTIZATION FOR COMPARISON

%

=====

=====

Pcm_min = min(x);

Pcm_max = max(x);

Delta_pcm = (pcm_max-pcm_min)/(L-1);

X_dpcm = round((x-pcm_min)/delta_pcm);

X_pcm = x_pcm*delta_pcm + pcm_min;

%%

=====

=====

% 5. ERROR CALCULATION

%

=====

=====

Pcm_error = x - x_pcm;

Dpcm_error = x - x_dpcm;

MSE_PCM = mean(pcm_error.^2);

MSE_DPCM = mean(dpcm_error.^2);

Fprintf('\n=====\\n');

```
Fprintf('PCM MSE = %.8f\n', MSE_PCM);  
Fprintf('DPCM MSE = %.8f\n', MSE_DPCM);  
Fprintf('=====\n');
```

```
%%
```

```
=====
```

```
=====
```

```
% 6. ORIGINAL AND PREDICTED SAMPLES
```

```
%
```

```
=====
```

```
=====
```

```
Figure;
```

```
Plot(t(1:150), x(1:150), 'LineWidth', 1.5);
```

```
Hold on;
```

```
Plot(t(1:150), x_pred(1:150), '—', 'LineWidth', 1.5);
```

```
Grid on;
```

```
Xlabel('Time (s)');
```

Ylabel('Amplitude');

Title('Original and Predicted Samples');

Legend('Original Signal',' Predicted Signal');

%%

=====

=====

% 7. PREDICTION ERROR

%

=====

=====

Figure;

Plot(t(1:150), prediction_error(1:150), 'LineWidth', 1.5);

Grid on;

Xlabel('Time (s)');

Ylabel('Prediction Error');

Title('DPCM Prediction Error');

```
%%
```

```
=====
```

```
=====
```

% 8. DELTA MODULATION FUNCTION

```
%
```

```
=====
```

```
=====
```

**Function [dm_bits, staircase, dm_error, mse,
step_change] = ...**

Delta_modulator(x, Delta)

N = length(x);

Dm_bits = zeros(1,N);

Staircase = zeros(1,N);

% Initial condition

Staircase(1) = 0;

For n = 2:N

If x(n) >= staircase(n-1)

Dm_bits(n) = 1;

Staircase(n) = staircase(n-1) + Delta;

Else

Dm_bits(n) = 0;

Staircase(n) = staircase(n-1) – Delta;

End

End

% Reconstruction error

Dm_error = x – staircase;

% Mean square error

Mse = mean(dm_error.^2);

% Change between consecutive staircase samples

Step_change = diff(staircase);

End

%%

=====

=====

% 9. DELTA MODULATION FOR DIFFERENT STEP SIZES

%

=====

=====

Delta_small = 0.05;

Delta_medium = 0.15;

Delta_large = 0.40;

[bits_s, stair_s, err_s, mse_s, change_s] = ...

Delta_modulator(x,Delta_small);

```
[bits_m, stair_m, err_m, mse_m, change_m] = ...
```

```
Delta_modulator(x,Delta_medium);
```

```
[bits_l, stair_l, err_l, mse_l, change_l] = ...
```

```
Delta_modulator(x,Delta_large);
```

```
%%
```

```
=====
```

```
=====
```

```
% 10. DELTA STAIRCASE PLOTS
```

```
%
```

```
=====
```

```
=====
```

```
Figure;
```

```
Plot(t(1:250),x(1:250),'LineWidth',1.5);
```

```
Hold on;
```

```
Stairs(t(1:250),stair_s(1:250),'LineWidth',1.2);
```

```
Grid on;  
Xlabel('Time (s)');  
Ylabel('Amplitude');  
Title('Delta Modulation: Small Step Size');  
Legend('Original Signal','DM Staircase');
```

```
Figure;
```

```
Plot(t(1:250),x(1:250),'LineWidth',1.5);  
Hold on;  
Stairs(t(1:250),stair_m(1:250),'LineWidth',1.2);
```

```
Grid on;  
Xlabel('Time (s)');  
Ylabel('Amplitude');  
Title('Delta Modulation: Moderate Step Size');  
Legend('Original Signal','DM Staircase');
```

```
Figure;
```



```
Plot(t(1:250),x(1:250),'LineWidth',1.5);  
Hold on;  
Stairs(t(1:250),stair_l(1:250),'LineWidth',1.2);
```

```
Grid on;  
Xlabel('Time (s)');  
Ylabel('Amplitude');  
Title('Delta Modulation: Large Step Size');  
Legend('Original Signal','DM Staircase');
```

```
%%  
=====
```

% 11. MSE VERSUS STEP SIZE

```
%  
=====
```

```
Delta_values = 0.01:0.01:0.50;
```

```
MSE_values = zeros(size(Delta_values));
```

```
For k = 1:length(Delta_values)
```

```
    [~,~,~,MSE_values(k),~] = ...
```

```
        Delta_modulator(x,Delta_values(k));
```

```
End
```

```
Figure;
```

```
Plot(Delta_values,MSE_values,'LineWidth',1.5);
```

```
Grid on;
```

```
Xlabel('Step Size \Delta');
```

```
Ylabel('Mean Square Error');
```

```
Title('MSE versus Step Size');
```

```
%%
```

```
=====
```

```
=====
```

```
% 12. SLOWLY VARYING INPUT
```

```
%
```

```
=====
```

```
=====
```

```
[~, stair_slow, ~, mse_slow, ~] = ...
```

```
Delta_modulator(x_slow,Delta_small);
```

```
Figure;
```

```
Plot(t(1:300),x_slow(1:300),'LineWidth',1.5);
```

```
Hold on;
```

```
Stairs(t(1:300),stair_slow(1:300),'LineWidth',1.2);
```

```
Grid on;
```

Xlabel('Time (s)');

Ylabel('Amplitude');

Title('DM for Slowly Varying Input');

Legend('Input','DM Staircase');

%%

=====

=====

% 13. RAPIDLY VARYING INPUT

%

=====

=====

[~, stair_fast, ~, mse_fast, ~] = ...

Delta_modulator(x_fast,Delta_small);

Figure;

```
Plot(t(1:250),x_fast(1:250),'LineWidth',1.5);
```

```
Hold on;
```

```
Stairs(t(1:250),stair_fast(1:250),'LineWidth',1.2);
```

```
Grid on;
```

```
Xlabel('Time (s)');
```

```
Ylabel('Amplitude');
```

```
Title('DM for Rapidly Varying Input');
```

```
Legend('Input','DM Staircase');
```

```
%%
```

```
=====
```

```
=====
```

```
% 14. MANDATORY +/- DELTA VALIDATION
```

%

=====
=====

Function validate_delta(staircase,Delta,name)

Changes = diff(staircase);

Tolerance = 1e-10;

Valid = all(...

Abs(changes-Delta)<tolerance | ...

Abs(changes+Delta)<tolerance | ...

Abs(changes)<tolerance);

% Zero changes may occur depending on initialization.

% For normal DM operation after the initial sample,

% all non-zero changes must be +Delta or -Delta.

Nonzero_changes = changes(abs(changes)>tolerance);

**Exact_plus = sum(abs(nonzero_changes-
Delta)<tolerance);**

**Exact_minus =
sum(abs(nonzero_changes+Delta)<tolerance);**

Fprintf('\n-----\n');

Fprintf('%s\n',name);

Fprintf('Step size Delta = %.4f\n',Delta);

Fprintf('Number of +Delta steps = %d\n',exact_plus);

Fprintf('Number of -Delta steps = %d\n',exact_minus);

If valid

**Fprintf('VALIDATION PASSED: Every non-zero step is
+Delta or -Delta.\n');**

Else

Fprintf('VALIDATION FAILED.\n');

End

End

```
Validate_delta(stair_s,Delta_small,'Small Delta');
```

```
Validate_delta(stair_m,Delta_medium,'Moderate Delta');
```

```
Validate_delta(stair_l,Delta_large,'Large Delta');
```

```
%%
```

```
=====
```

```
=====
```

```
% 15. DISPLAY MSE RESULTS
```

```
%
```

```
=====
```

```
=====
```

```
Fprintf('\n=====  
====\n');
```

```
Fprintf('DELTA MODULATION RESULTS\n');
```

```
Fprintf('=====  
====\n');
```

```
Fprintf('Small Delta  = %.2f, MSE = %.8f\n', ...
```

```
Delta_small,mse_s);
```



```
Fprintf('Medium Delta  = %.2f, MSE = %.8f\n', ...  
Delta_medium, mse_m);
```

```
Fprintf('Large Delta   = %.2f, MSE = %.8f\n', ...  
Delta_large, mse_l);
```

```
Fprintf('Slow input MSE = %.8f\n', mse_slow);
```

```
Fprintf('Fast input MSE = %.8f\n', mse_fast);
```

```
Fprintf('=====  
====\n');
```

7. Expected Graphs

Your MATLAB program will produce the required visualizations:

Graph 1 – Original vs Predicted Samples

The predicted signal follows the previous sample of the original signal.

Graph 2 – Prediction Error

$$e[n] = x[n] - x[n - 1]$$

The error signal has considerably smaller amplitude than the original signal for a highly correlated input.

Graph 3 – Delta Staircase

The reconstructed DM signal appears as a staircase.

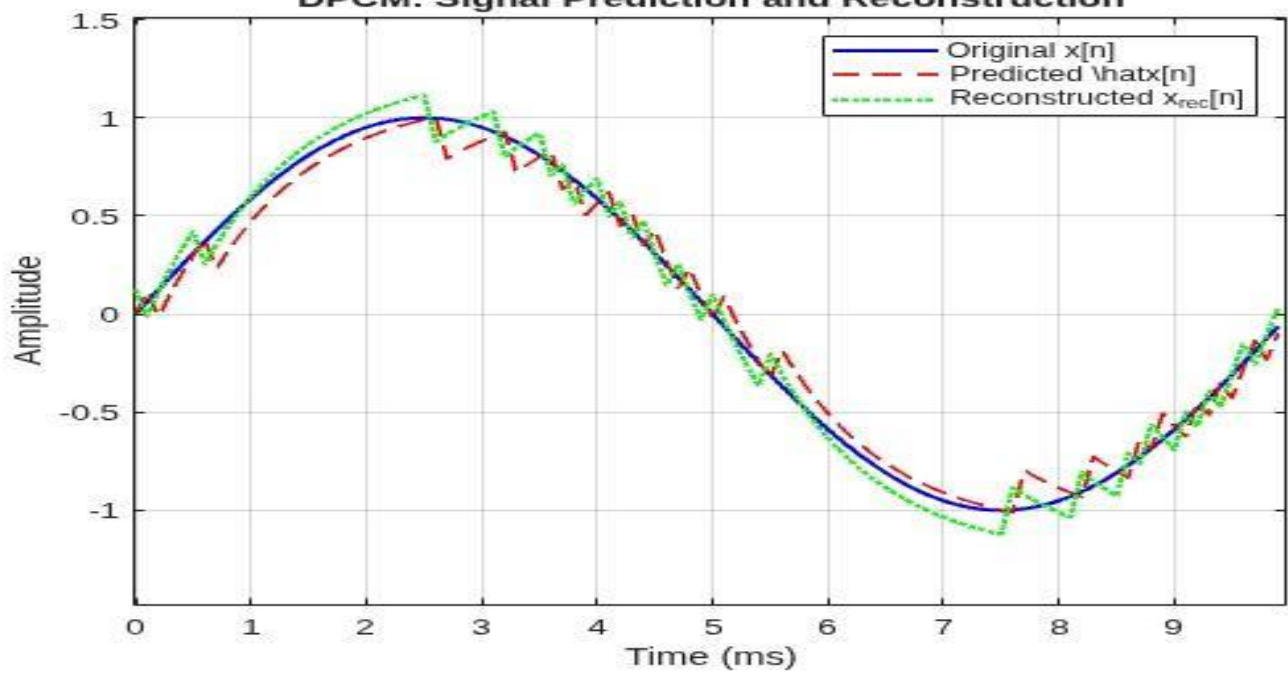
Each step must satisfy:

$$\Delta x_r = \pm \Delta$$

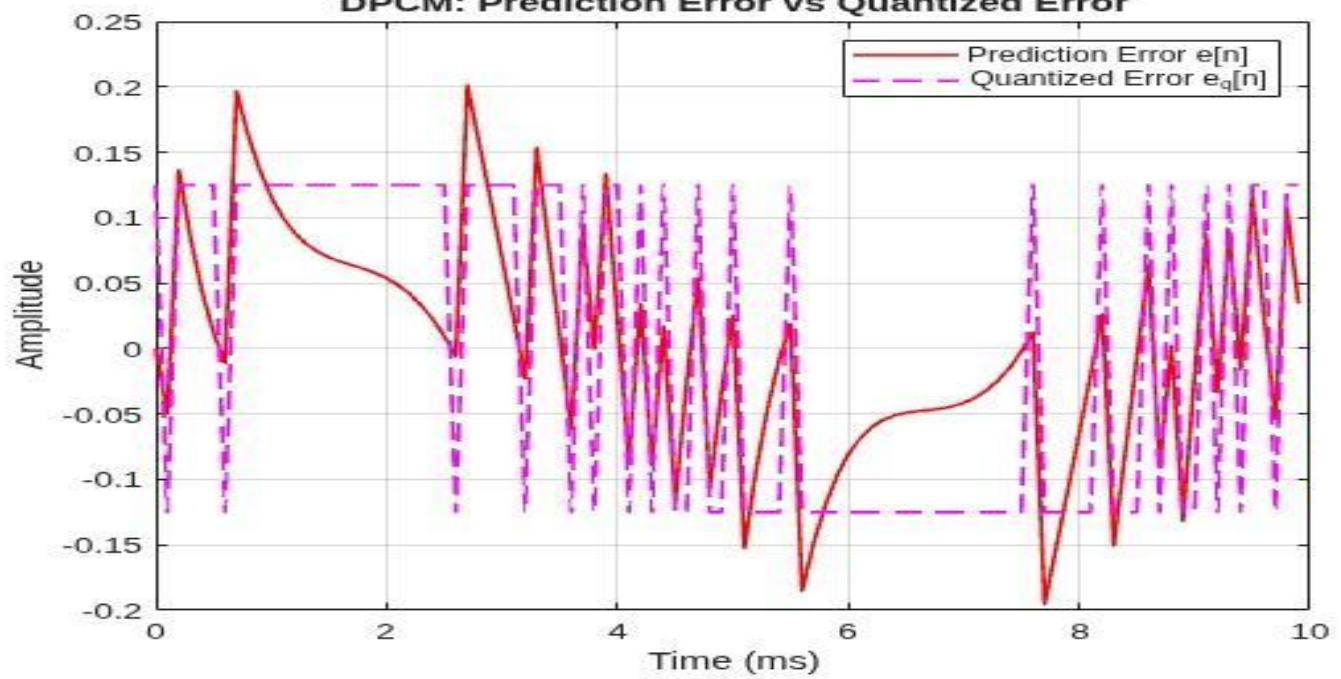
Graph 4 – MSE vs Step Size

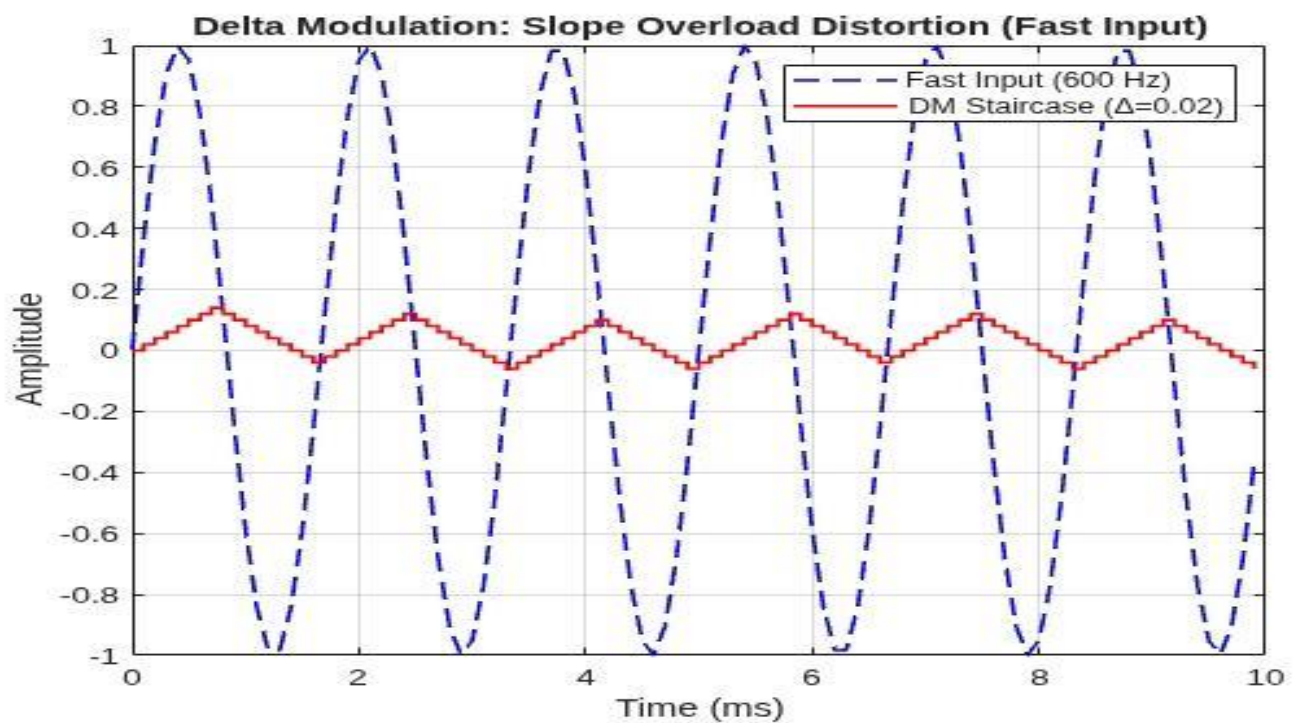
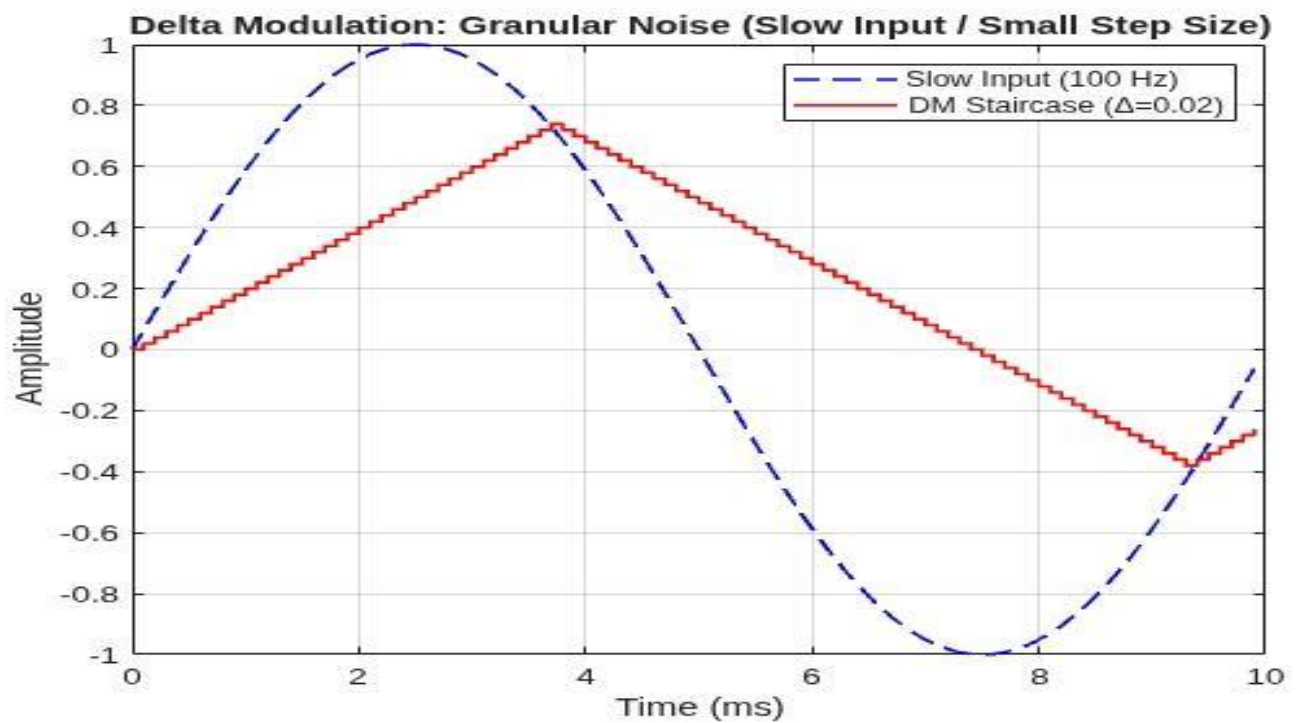
This graph demonstrates the trade-off between step size and reconstruction error.

DPCM: Signal Prediction and Reconstruction



DPCM: Prediction Error vs Quantized Error





12. DPCM Prediction and Quantization Error

For DPCM:

$$e_p[n] = x[n] - \hat{x}[n]$$

The prediction error is generally smaller than the original signal because adjacent samples of a slowly changing signal are strongly correlated.

After quantization:

$$e_q[n] = e_p[n] - e_{pq}[n]$$

where $e_{pq}[n]$ represents the quantized prediction error.

Thus DPCM improves efficiency by quantizing the **difference signal rather than the complete signal**.

13. Mandatory Validation

The assessment specifically requires:

Inspect whether each delta-modulator output step changes by exactly $+\Delta$ or $-\Delta$.

14. Observation and Interpretation

Before simulation

Small step size:

The staircase is expected to follow slowly varying signals well, but it may not track rapidly varying signals.

Moderate step size:

A reasonable balance between tracking ability and granular noise is expected.

Large step size:

The staircase should follow rapidly changing signals better, but the reconstructed signal is expected to show larger granular variations.

After simulation

The expected results should agree with the theory:

- DPCM reduces sample redundancy using prediction.
- The prediction error has a smaller range than the original signal for correlated samples.
- Small Δ can produce slope-overload distortion for rapidly varying signals.
- Large Δ can produce granular noise.
- DM staircase transitions occur by exactly $+\Delta$ or $-\Delta$.

15. Result

The DPCM and Delta Modulation systems were successfully implemented in MATLAB.

The first-order DPCM predictor generated a prediction error by subtracting the previous sample from the current sample. Quantization and reconstruction were performed separately.

Delta Modulation was implemented for small, moderate and large step sizes. The effect of step size and input frequency on reconstruction quality was observed. **Granular noise** was associated with excessive step size, while **slope-overload distortion** occurred when the staircase could not follow a rapidly varying input.

The mandatory validation confirmed that each non-zero Delta Modulation staircase transition is exactly:

$$+\Delta \text{ or } -\Delta$$

Therefore, the experimental results agree with the theoretical behavior of DPCM and Delta Modulation.

16. Conclusion

DPCM effectively reduces sample redundancy by transmitting the quantized prediction error instead of the complete sample value. A first-order predictor provides a simple and effective prediction for correlated signals.

Delta Modulation further simplifies the system by representing each difference using one bit. However, its performance strongly depends on the step size. A very small step size can cause **slope overload**, whereas a very large step size can produce **granular noise**.

Hence, an appropriate value of Δ is essential for obtaining good reconstruction quality.

THANK YOU..!